

第10章 控制单元的设计

10.1 组合逻辑设计

10.2 微程序设计



内容回顾:

- 计算机如何执行程序：控制单元发出各种微操作命令（控制信号）序列；控制计算机所有部件有次序地完成相应的操作，以达到执行程序的目的
- 不同的机器指令对应不同的微操作命令（控制信号）序列；完成不同指令的过程中，有些操作是相同的或者相似的

- 机器周期是所有指令执行过程中的一个基准时间，
一个机器周期内可完成若干个（关联的）微操作
- 时钟周期是控制计算机操作的最小单位时间，在
一个时钟周期可完成一个或几个需要同时执行的操作
- 一个指令周期包含若干个机器周期，一个机器周期包含若干个时钟周期或节拍
- 控制单元的每一个控制信号都是在指定机器周期内的指定时刻（节拍）发出的

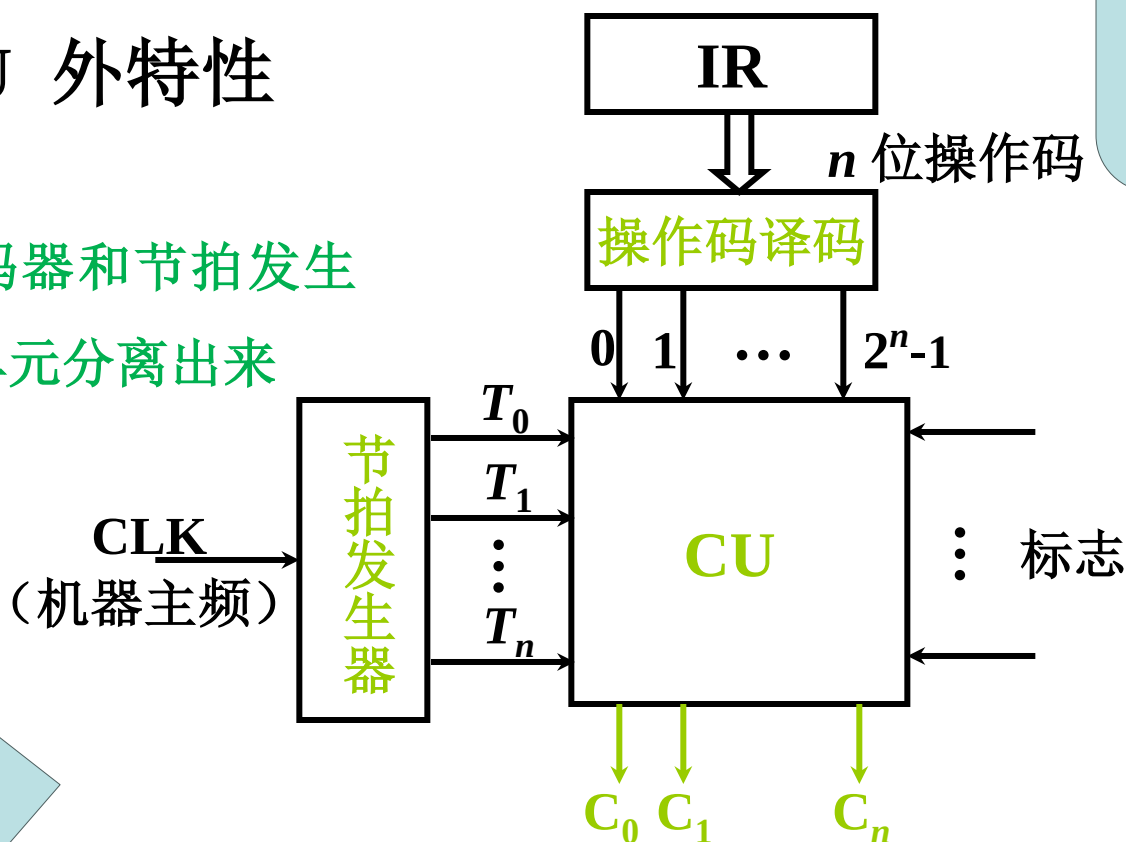
10.1 组合逻辑设计

一、组合逻辑控制单元框图

1. CU 外特性

把指令译码器和节拍发生器
器从控制单元分离出来

控制单元利用节拍发生器的驱动，给这组微操作命令加上时序，成为微操作命令序列



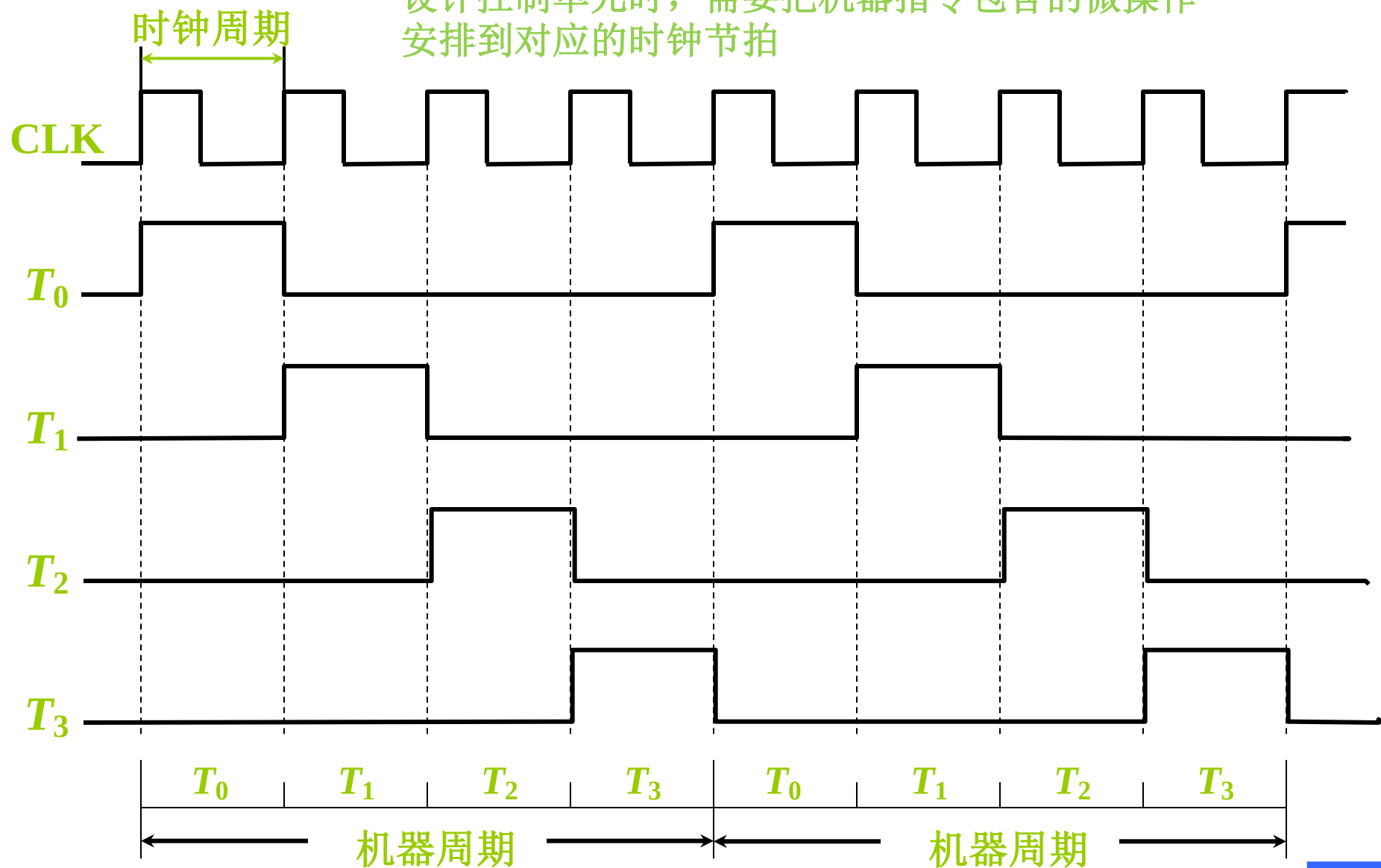
操作码译码
把指令翻译成
一组微操作
命令

至CPU内部及系统总线的控制信号



2. 节拍信号 控制单元按照一定的节拍发出各种控制信号

设计控制单元时，需要把机器指令包含的微操作安排到对应的时钟节拍

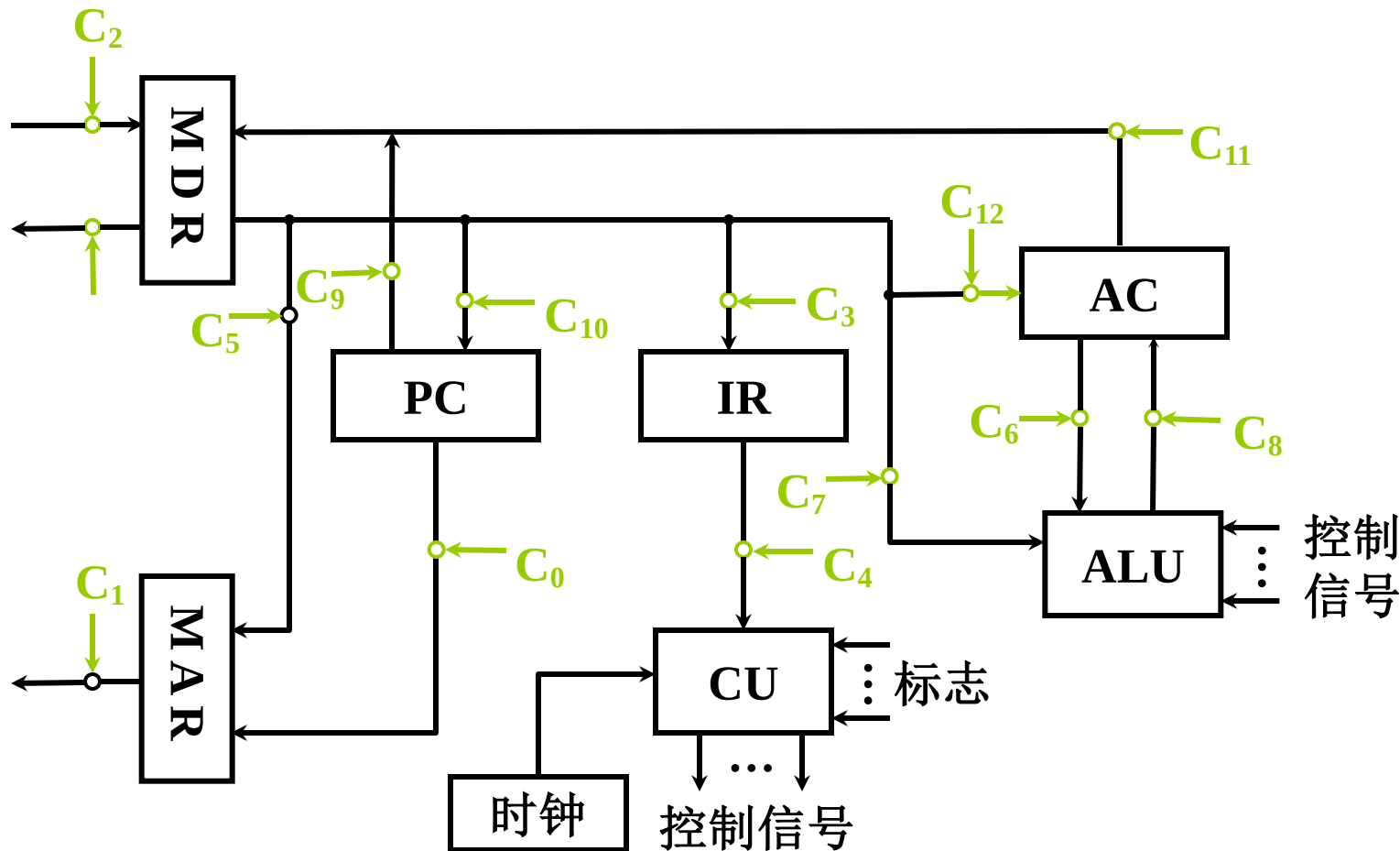


二、微操作的节拍安排

CU采用 同步控制方式形成微操作序列

假设一个机器周期内有 3 个节拍（时钟周期）

CPU 内部结构采用非总线方式（如下图），IR和MAR之间有数据通路（下图未画出）



1. 安排微操作时序的原则

原则一 微操作的 先后顺序不得 随意 更改

原则二 被控对象不同 的微操作

尽量安排在 一个节拍 内完成

原则三 占用 时间较短 的微操作

尽量 安排在一个节拍 内完成

并允许有先后顺序



2. 取指周期 微操作的 节拍安排

T_0 $PC \longrightarrow MAR$

原则二

$1 \longrightarrow R$

T_1 $M (MAR) \longrightarrow MDR$

原则二

$(PC) + 1 \longrightarrow PC$

T_2 $MDR \longrightarrow IR$

原则三

$OP (IR) \longrightarrow ID$

3. 间址周期 微操作的 节拍安排

T_0 $Ad (IR) \longrightarrow MAR$

$1 \longrightarrow R$

T_1 $M (MAR) \longrightarrow MDR$

T_2 $MDR \longrightarrow Ad (IR)$



4. 执行周期 微操作的节拍安排

① CLA T_0

T_1

T_2 $0 \longrightarrow AC$

② COM T_0

T_1

T_2 $\overline{AC} \longrightarrow AC$

③ SHR T_0

T_1

T_2 $L(AC) \longrightarrow R(AC)$

$AC_0 \longrightarrow AC_0$



④ CSL T_0
 T_1
 T_2 $R(AC) \longrightarrow L(AC)$ $AC_0 \longrightarrow AC_n$

⑤ STP T_0
 T_1
 T_2 $0 \longrightarrow G$

⑥ ADD X T_0 $Ad(IR) \longrightarrow MAR$ $1 \longrightarrow R$
 T_1 $M(MAR) \longrightarrow MDR$
 T_2 $(AC) + (MDR) \longrightarrow AC$

⑦ STA X T_0 $Ad(IR) \longrightarrow MAR$ $1 \longrightarrow W$
 T_1 $AC \longrightarrow MDR$
 T_2 $MDR \longrightarrow M(MAR)$



⑧ LDA X T_0 $Ad(IR) \longrightarrow MAR \quad 1 \longrightarrow R$

T_1 $M(MAR) \longrightarrow MDR$

T_2 $MDR \longrightarrow AC$

⑨ JMP X

T_0 访存指令在执行周期的 T_0 、 T_1 节拍进行访
存操作，非访存指令和转移类指令在这两个
节拍可不安排操作；对所有指令，在 T_2 节
拍安排关键操作，体现了指令的主要内容

T_1

T_2 $Ad(IR) \longrightarrow PC$

⑩ BAN X

T_0

T_1

T_2 $A_0 \cdot Ad(IR) + \bar{A}_0 \cdot PC \longrightarrow PC$



5. 中断周期 微操作的 节拍安排

T_0 $0 \longrightarrow \text{MAR}$ $1 \longrightarrow \text{W}$ 硬件关中断

T_1 $\text{PC} \longrightarrow \text{MDR}$

T_2 $\text{MDR} \longrightarrow \text{M}(\text{MAR})$ 向量地址 $\longrightarrow \text{PC}$

由中断隐指令完成上述操作



三、组合逻辑设计步骤

根据前面给出的微操作节拍安排, 组合逻辑设计控制单元步骤是:

- 1) 列出微操作命令的操作时间表
- 2) 写出微操作命令的最简逻辑表达式
- 3) 画出微操作命令的逻辑电路图

每个微操作命令对应一个时序逻辑电路

三、组合逻辑设计步骤

1. 列出操作时间表

这里只给出6条指令/教材上10条

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		$PC \rightarrow MAR$						
			$1 \rightarrow R$						
	T_1		$M(MAR) \rightarrow MDR$						
			$(PC) + 1 \rightarrow PC$						
	T_2		$MDR \rightarrow IR$						
			$OP(IR) \rightarrow ID$						
		I	$1 \rightarrow IND$						
		$\bar{I}$	$1 \rightarrow EX$						

间址标志，若I=1，则进入间址周期；若I=0，则进入执行周期



操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) $\rightarrow$ MAR						
			1 $\rightarrow$ R						
	T_1		M(MAR) $\rightarrow$ MDR						
	T_2		MDR $\rightarrow$ Ad (IR)						
		$\overline{\text{IND}}$	1 $\rightarrow$ EX						

间址周期标志，若IND为0，则进入执行周期



操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		Ad (IR) $\rightarrow$ MAR						
			1 $\rightarrow$ R						
			1 $\rightarrow$ W						
	T_1		M(MAR) $\rightarrow$ MDR						
			AC $\rightarrow$ MDR						
	T_2		(AC)+(MDR) $\rightarrow$ AC						
			MDR $\rightarrow$ M(MAR)						
			MDR $\rightarrow$ AC						
			0 $\rightarrow$ AC						



操作时间表 体现某条指令在某个节拍有某个操作

工作周期 标记	节拍	状态 条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		$PC \rightarrow MAR$	1	1	1	1	1	1
			$1 \rightarrow R$	1	1	1	1	1	1
	T_1		$M(MAR) \rightarrow MDR$	1	1	1	1	1	1
			$(PC) + 1 \rightarrow PC$	1	1	1	1	1	1
	T_2		$MDR \rightarrow IR$	1	1	1	1	1	1
			$OP(IR) \rightarrow ID$	1	1	1	1	1	1
		I	$1 \rightarrow IND$			1	1	1	1
		$\bar{I}$	$1 \rightarrow EX$	1	1	1	1	1	1



操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) $\rightarrow$ MAR			1	1	1	1
			1 $\rightarrow$ R			1	1	1	1
	T_1		M(MAR) $\rightarrow$ MDR			1	1	1	1
	T_2		MDR $\rightarrow$ Ad (IR)			1	1	1	1
		$\overline{\text{IND}}$	1 $\rightarrow$ EX			1	1	1	1



操作时间表

执行阶段对应COM/JMP指令的微操作未列出

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		Ad (IR) $\rightarrow$ MAR			1	1	1	
			1 $\rightarrow$ R			1		1	
			1 $\rightarrow$ W				1		
	T_1		M(MAR) $\rightarrow$ MDR			1		1	
			AC $\rightarrow$ MDR				1		
	T_2		(AC)+(MDR) $\rightarrow$ AC			1			
			MDR $\rightarrow$ M(MAR)				1		
			MDR $\rightarrow$ AC					1	
			0 $\rightarrow$ AC	1					



2. 写出微操作命令的最简表达式

$M(MAR) \longrightarrow MDR$

以这个操作为例
按照书上10条指令给出

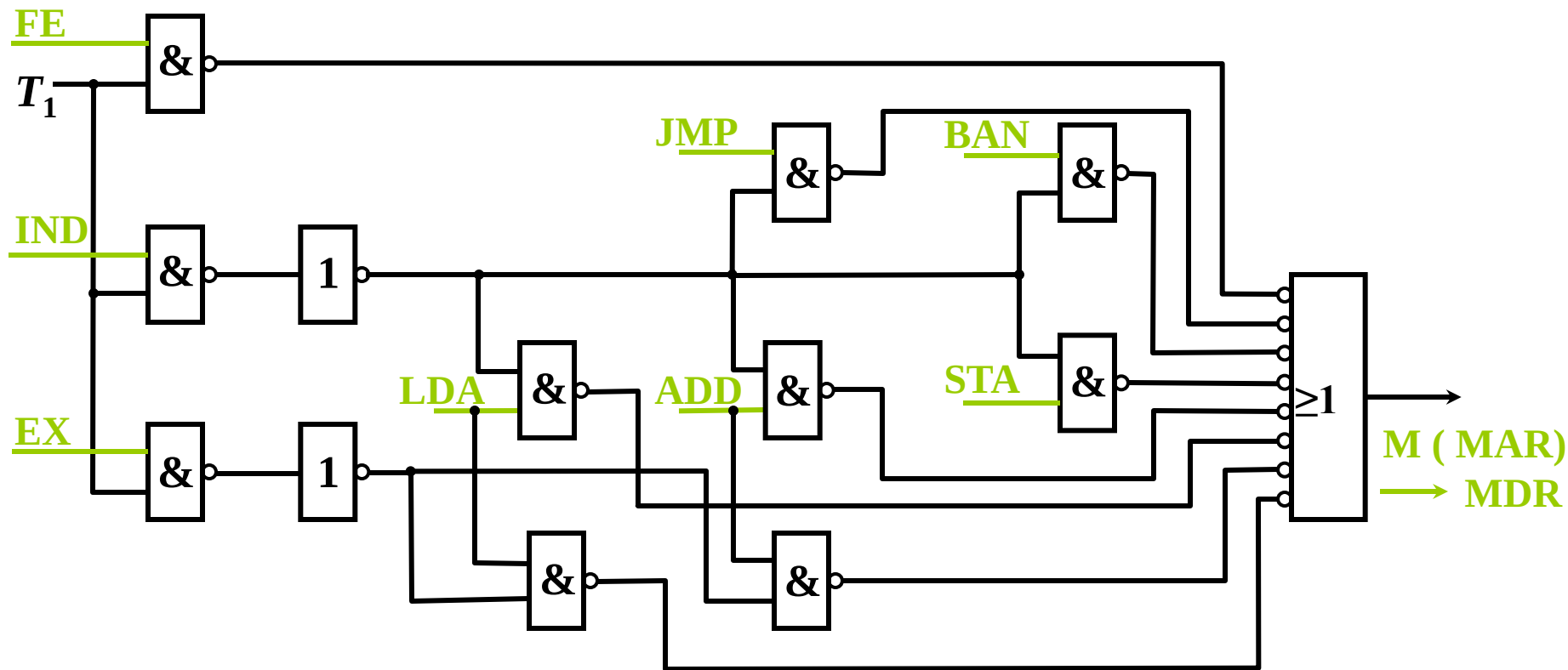
$$= FE \cdot T_1 + IND \cdot T_1 (ADD + STA + LDA + JMP + BAN) \\ + EX \cdot T_1 (ADD + LDA)$$

$$= T_1 \{ FE + IND (ADD + STA + LDA + JMP + BAN) \\ + EX (ADD + LDA) \}$$

ADD、STA、LDA等均来自操作码译码器的输出。



3. 画出逻辑图



特点

- 思路清晰，简单明了
- 每个微操作命令对应一个逻辑电路，线路结构庞杂，指令系统越全，调试修改困难
- 速度快 RISC 处理器适用

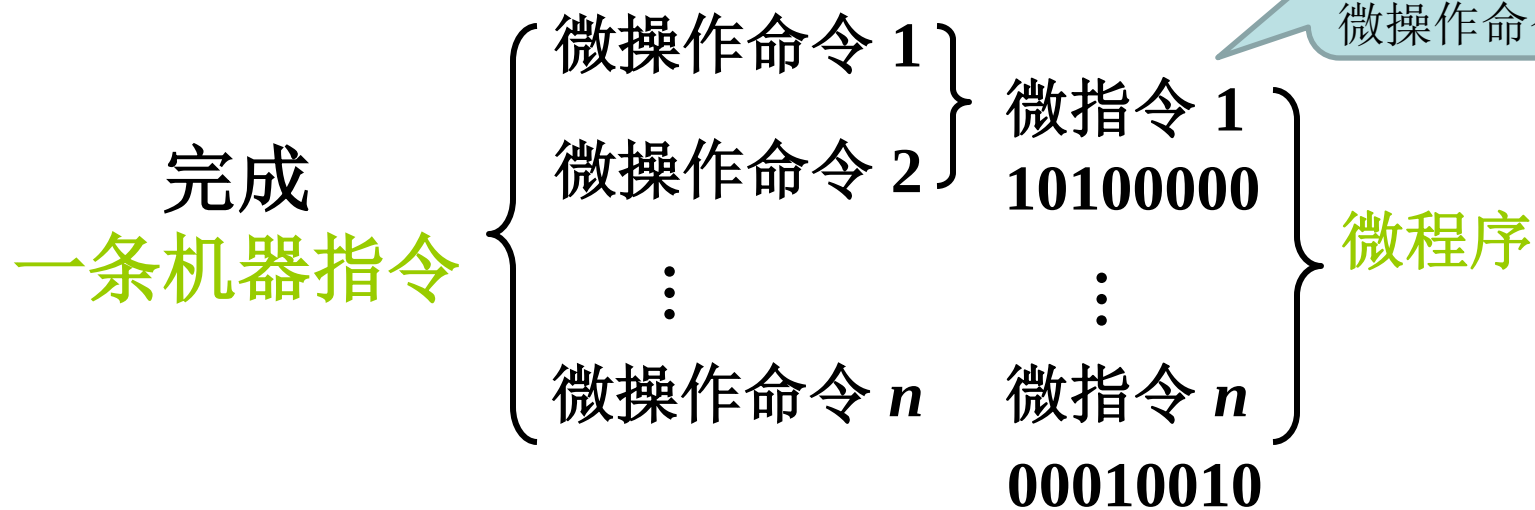


10.2 微程序设计

一、微程序设计思想的产生

组合逻辑控制单元线路庞杂，因此采用与存储程序类似的方法，解决微操作命令序列的形成

1951 英国剑桥大学教授 Wilkes 提出



一条机器指令对应一个微程序 存入控制存储器 ROM

用寻找用户程序的机器指令的方法寻找微程序中的微指令



微指令以二进制代码形式表示，每位代表一个控制信号；逐条执行每一条微指令，也就完成了一条机器指令的全部操作

微程序控制单元的核心部件是控制存储器，执行一条机器指令必须多次访问控制存储器，以取出多条微指令来执行各个微操作，控制存储器要求速度够高

微程序设计方式只需修改微指令的二进制代码，就可以改变操作内容，便于调试、修改甚至增删机器指令

二、微程序控制单元框图及工作原理

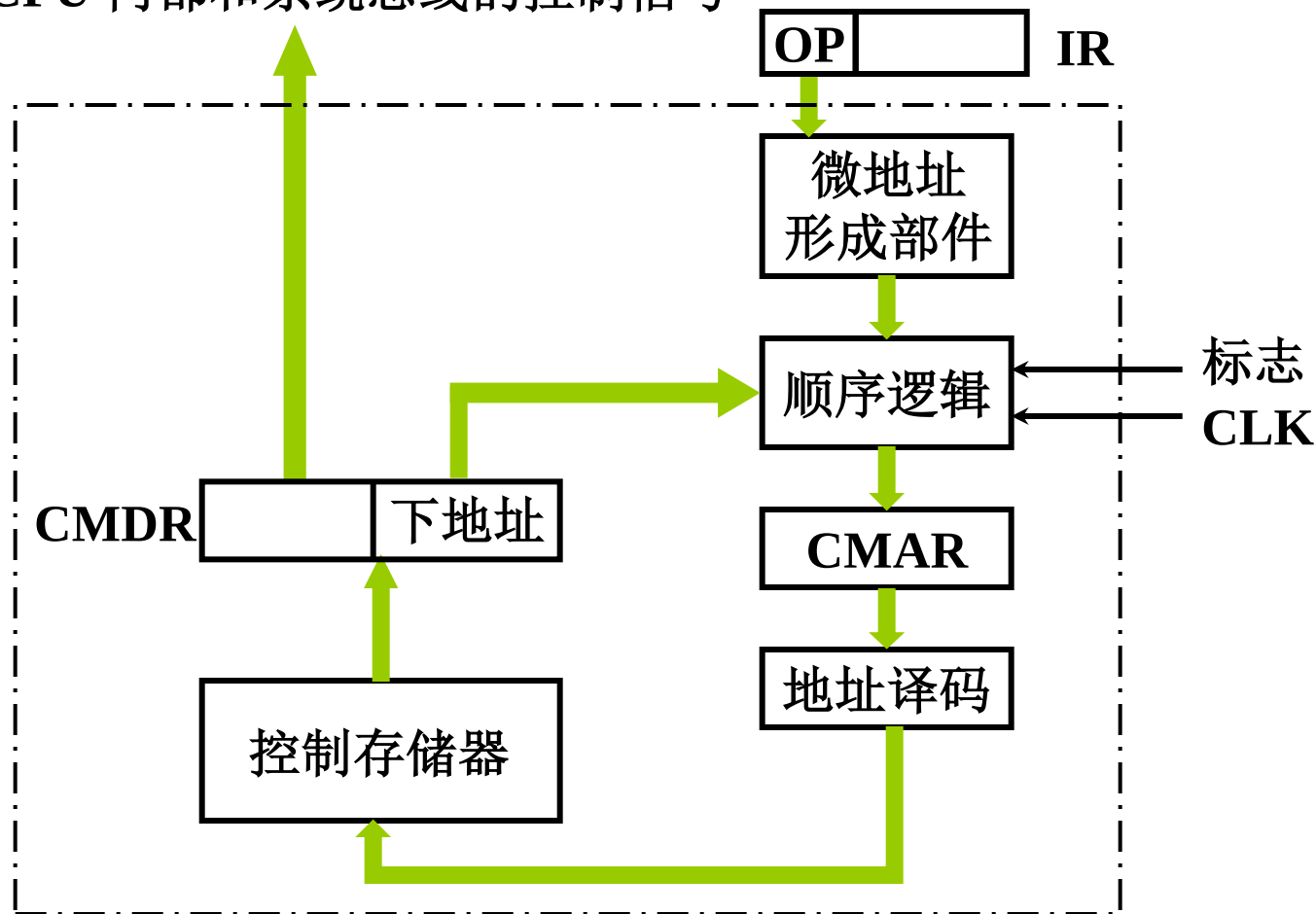
- 采用微程序设计方法设计控制单元的过程就是编写每一条机器指令对应的微程序，按照执行一条机器指令所需的微操作命令的先后顺序编写
- 任何一条机器指令的取指周期的操作都一样，因此把取指令操作的命令统一编成一个微程序，取指微程序负责将指令从主存单元取出送至指令寄存器，间址周期、中断周期的微程序也是如此统一处理
- 控制存储器中微程序的个数应为机器指令数再加上对应取指、间址寻址和中断周期的3个微程序

1. 机器指令对应的微程序

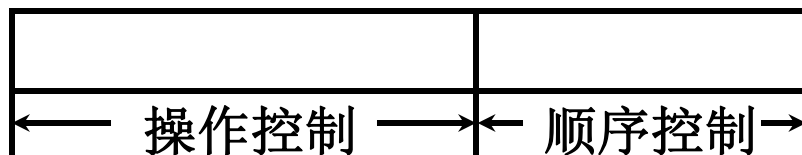


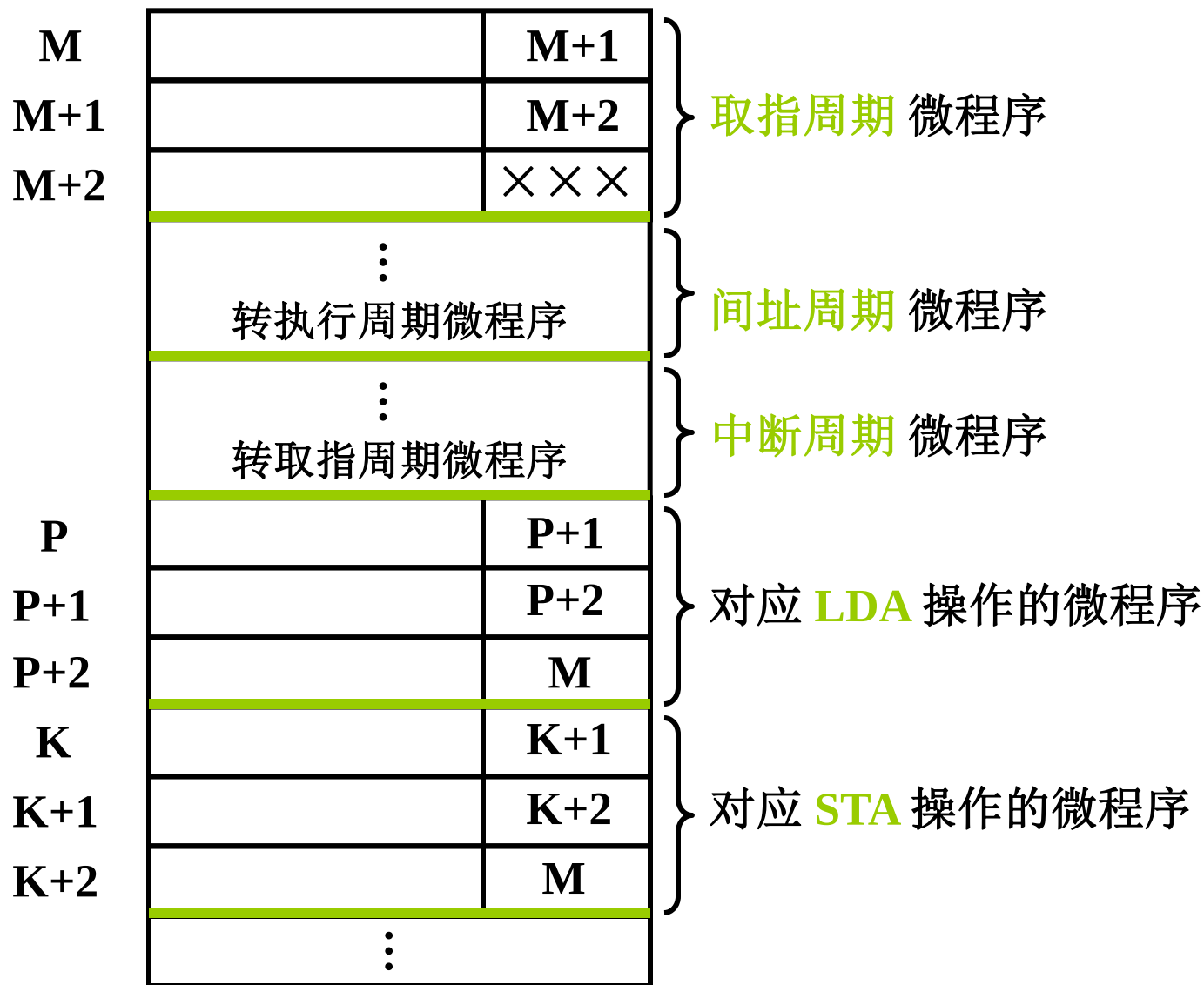
2. 微程序控制单元的基本框图

至 CPU 内部和系统总线的控制信号

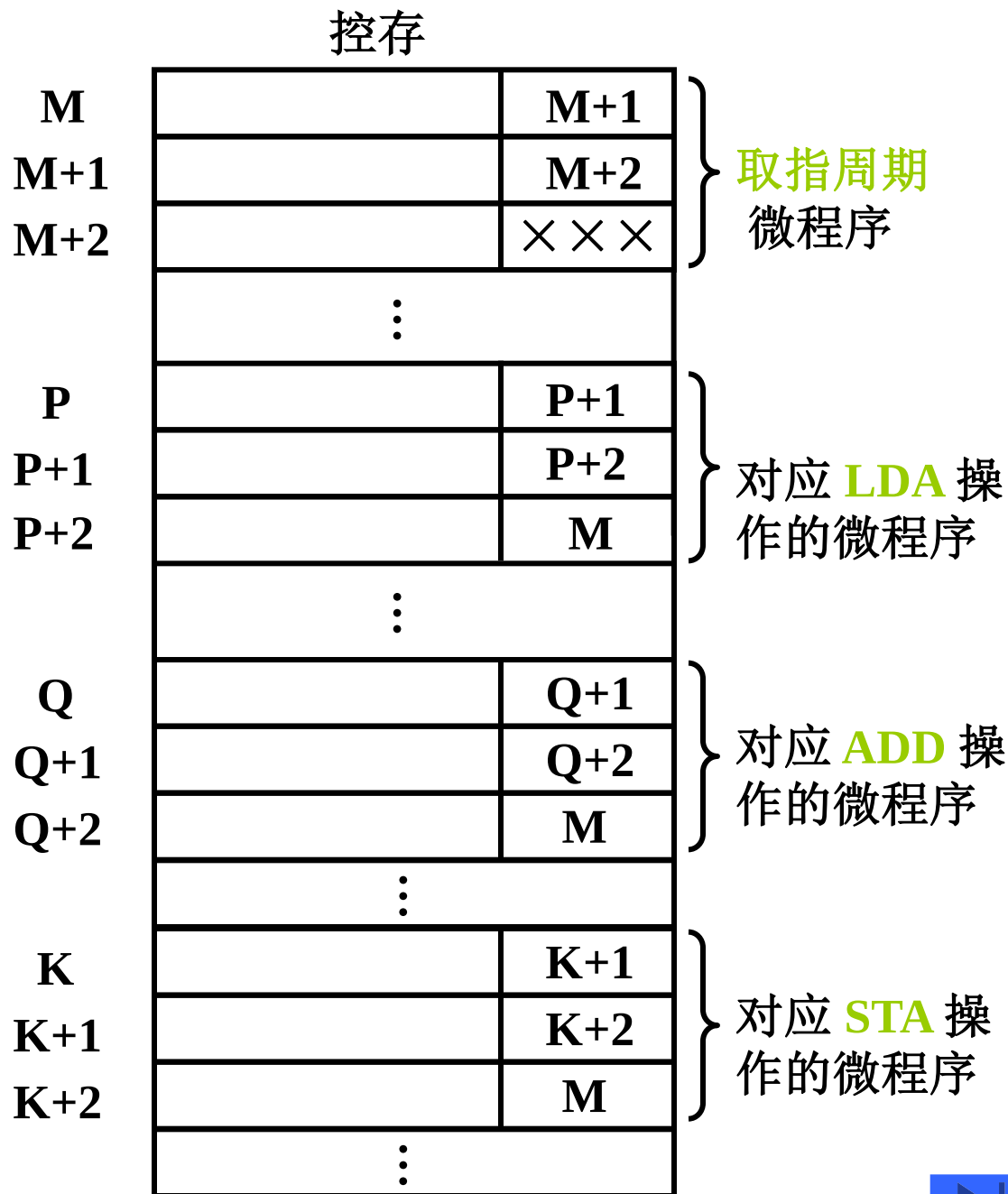
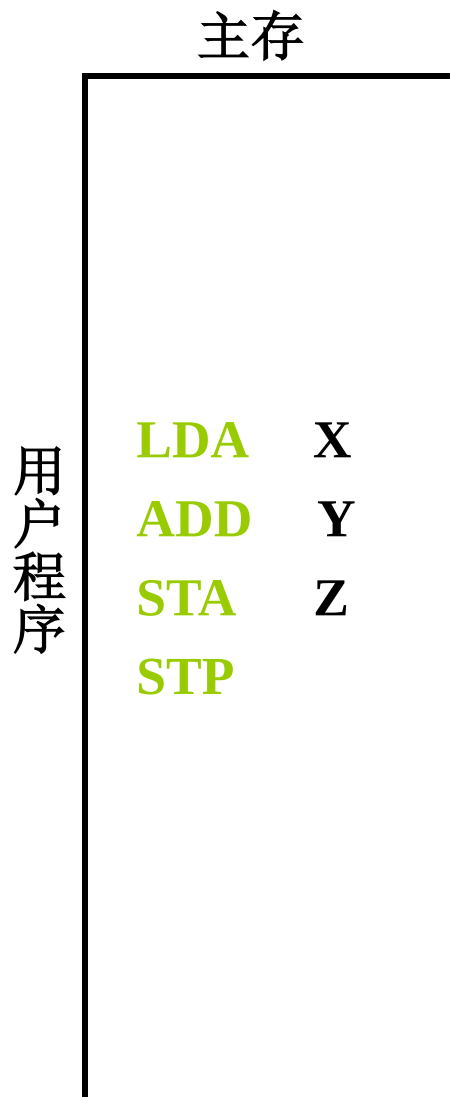


微指令基本格式





3. 工作原理



(1) 取指阶段

执行取指微程序

$M \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

形成下条微指令地址 $M + 1$

$Ad(CMDR) \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

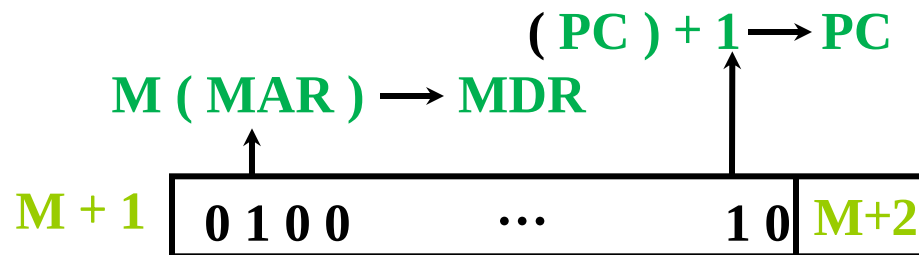
由 CMDR 发命令

形成下条微指令地址 $M + 2$

$Ad(CMDR) \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令



(2) 执行阶段 执行 LDA 微程序

$OP(IR) \longrightarrow \text{微地址形成部件} \longrightarrow CMAR \quad (P \longrightarrow CMAR)$

$CM(CMAR) \longrightarrow CMDR$

由 CMDR 发命令

形成下条微指令地址
 $Ad(CMDR) \longrightarrow CMAR$

$CM(CMAR) \longrightarrow CMDR$

由 CMDR 发命令

形成下条微指令地址
 $Ad(CMDR) \longrightarrow CMAR$

$CM(CMAR) \longrightarrow CMDR$

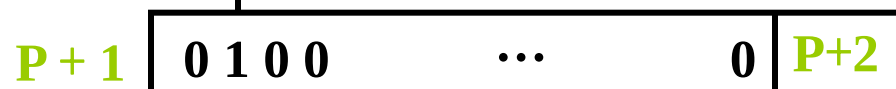
由 CMDR 发命令

形成下条微指令地址
 $Ad(CMDR) \longrightarrow CMAR$

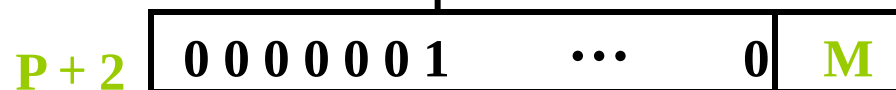
$Ad(IR) \longrightarrow MAR \quad 1 \longrightarrow R$



$M(MAR) \longrightarrow MDR$



$MDR \longrightarrow AC$



$(M \longrightarrow CMAR)$

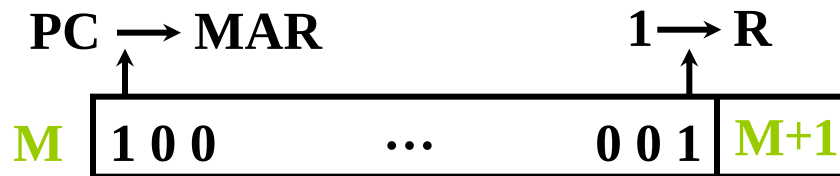


(3) 取指阶段 执行取指微程序

$M \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令



⋮

全部微指令都保存在 CM 中，程序执行过程中只需将其读出：通过逐条形成下一条微指令地址、取出下一条微指令、发出各种微操作命令，实现从主存逐条取出、分析并执行机器指令

- 关键
- 微指令的操作控制字段如何形成微操作命令
 - 微指令的后续地址如何形成

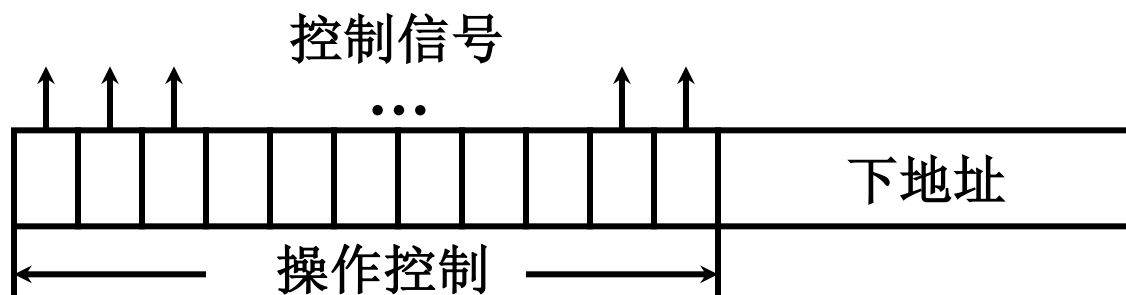


三、微指令的编码方式（控制方式）

如何对操作控制字段进行编码，以形成控制信号

1. 直接编码（直接控制）方式

在微指令的操作控制字段中，**每一位代表一个微操作命令**；只要从控存读出微指令，即可由控制字段发出控制命令



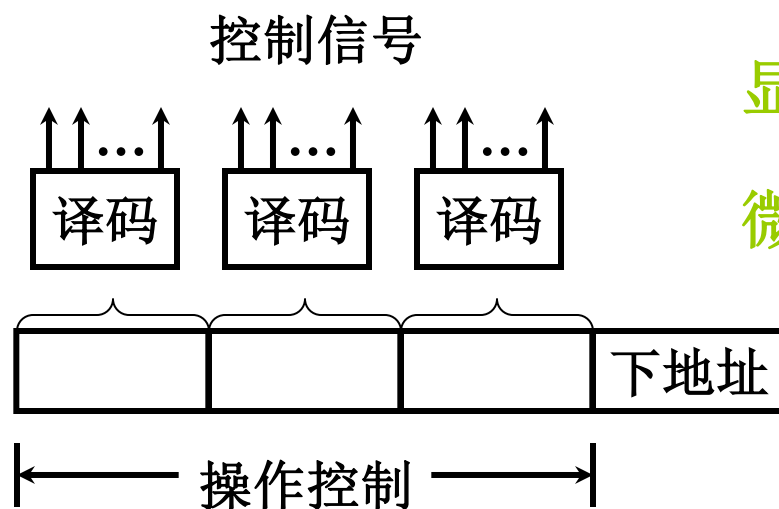
速度最快
控存容量大

某位为 “1” 表示该控制信号有效



2. 字段直接编码方式

将微指令的控制字段分成若干“段”，每段经译码后发出一个控制信号；分段数与需并行发出的命令个数有关



显式编码

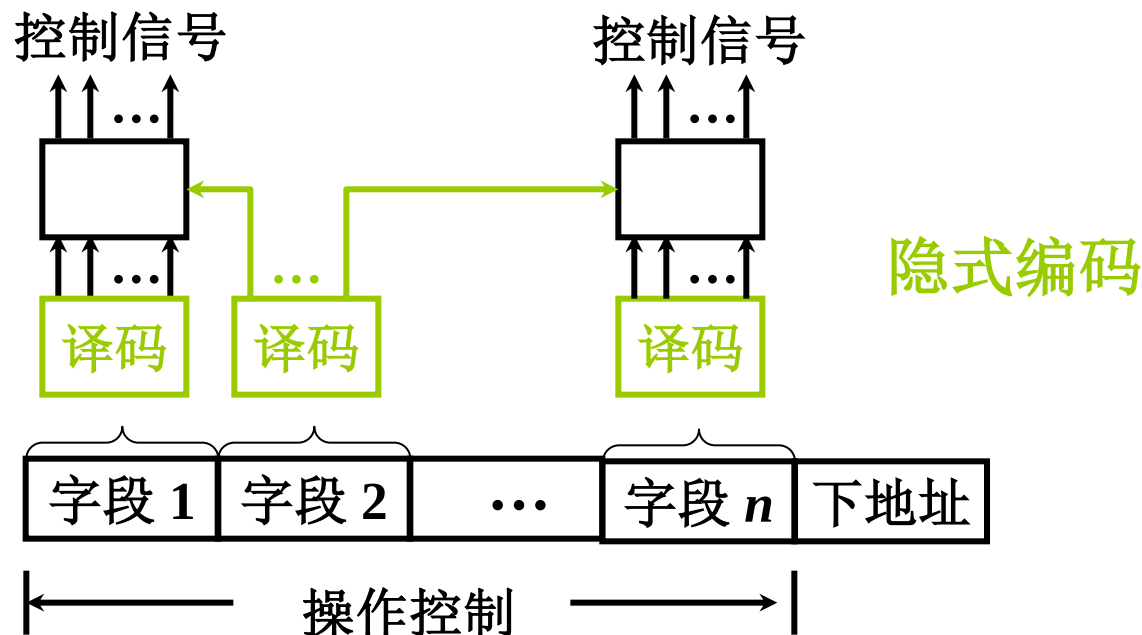
微程序执行速度较慢

每个字段中的命令是互斥（不能同时发出）的，每段的长度与要求互斥的命令个数有关

缩短了微指令字长，增加了译码时间



3. 字段间接编码方式



4. 混合编码

直接编码和字段编码（直接和间接）混合使用

5. 其他

微指令中还可以设置常数字段，用于提供常数、计数器初值等



四、微指令序列地址的形成

1. 微指令的 **下地址字段** 指出
2. 根据机器指令的 **操作码** 形成
3. **增量计数器**

$$(CMAR) + 1 \rightarrow CMAR$$

4. **分支转移**

操作控制字段	转移方式	转移地址
--------	------	------

转移方式 指明判别条件

转移地址 指明转移成功后的去向



5. 通过测试网络形成微指令地址

例如：微指令高段地址，由微指令的高段地址码直接形成，
低段地址，由微指令的低段地址码通过测试网络形成

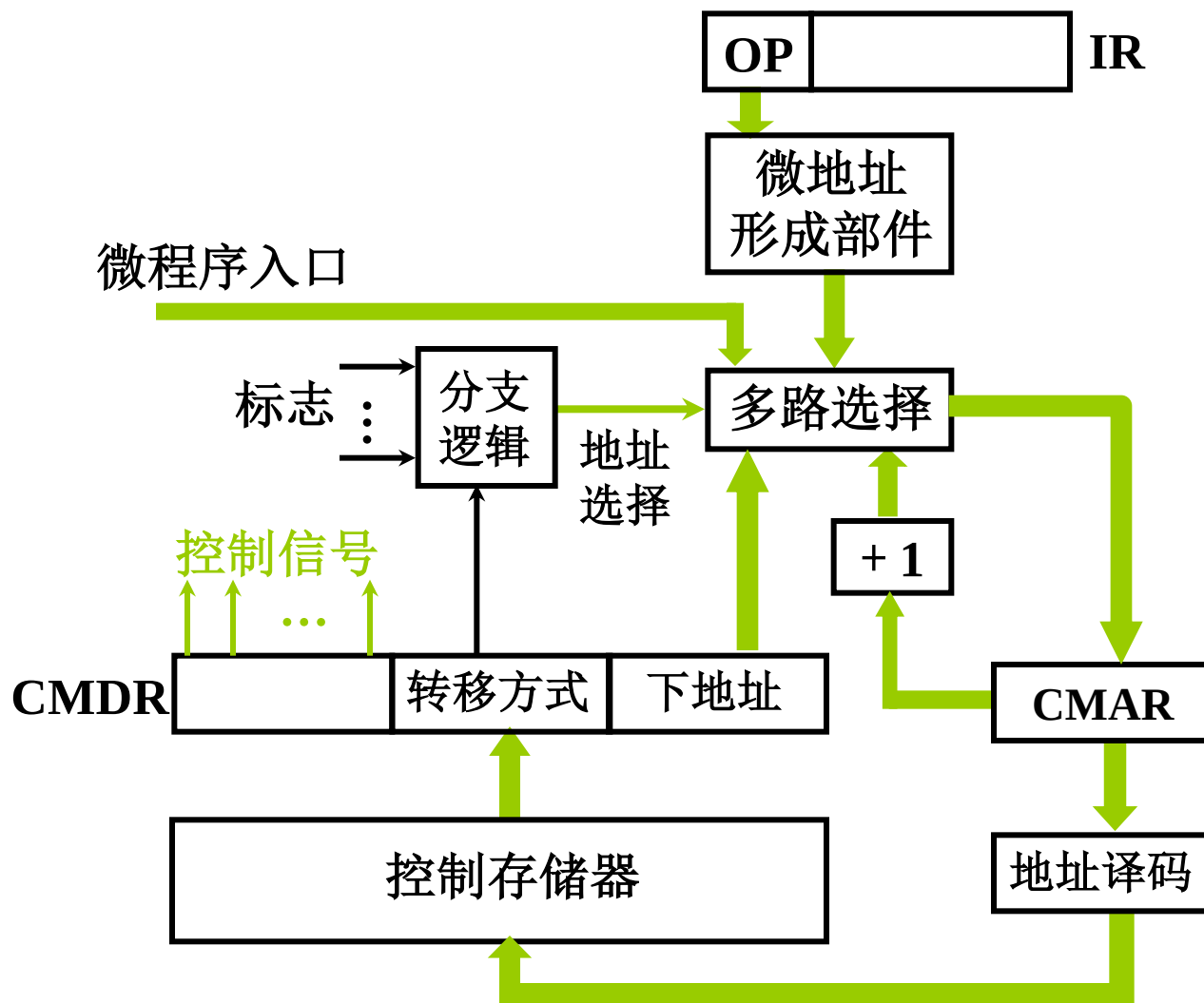
6. 由硬件产生微程序入口地址

开机后第一条微指令地址，由专门硬件产生，即
取指周期微程序入口地址

中断周期，由硬件产生中断周期微程序首地址
间址周期微程序入口地址，也可由硬件产生



10. 后续微指令地址形成方式原理图



例：某处理器共有52个微操作控制信号，构成5个互斥的微命令组，各组分别包括5、8、2、15、22个微命令，已知可判定的外部条件有2个，微指令字长为28位。

- (1) 设计微指令格式，要求微指令的下地址字段直接给出后续微指令的地址
- (2) 指出控制存储器的容量

(1) 有5个互斥的微命令组，每组包含若干位，每组译码的结果应该能表示本组包含的所有微命令加上一个不发命令的情况，则5个控制字段分别需要给出6、9、3、16、23种状态，对应着3、4、2、4、5这几种字段位数（共18位），条件测试字段取2位，这些加起来是20位，因为微指令字长为28位，因此下地址字段为8位。

(2) 因为下地址字段为8位，可表示256个微指令，所以控制存储器容量为 256×28 位。

Review

- 计算机如何执行程序：控制单元发出各种微操作命令（控制信号）序列；控制计算机所有部件有次序地完成相应的操作，以达到执行程序的目的
- 微程序设计控制单元的思想：把一个机器指令编写成一个微程序，一个微程序包含若干条微指令，每条微指令对应一个或者几个微操作命令
- 微程序存到一个控制存储器中，逐条执行每一条微指令，相应地就完成了一条机器指令的全部操作

- 微指令以二进制代码形式表示，微指令的基本格式包含一个操作控制字段和一个顺序控制字段，由操作控制字段发出各种控制信号，由下地址字段指出下一条微指令的地址
- 对于微程序控制单元设计，关键是如何由微指令的操作控制字段形成微操作命令，以及如何形成下一条微指令的地址

九、微程序设计举例

- 用微程序法设计控制单元，主要任务是编写对应各条机器指令的微程序
- 微程序法设计控制单元的步骤：
 1. 写出对应机器指令的全部微操作及节拍安排
 2. 确定微指令格式
 3. 编写出每条微指令的二进制代码(微指令码点)

微程序设计举例

1. 写出对应机器指令的微操作及节拍安排

假设 CPU 结构与组合逻辑相同，仅考虑取指、执行阶段

(1) 取指阶段微操作分析 一个T内的微操作安排一个微指令完成，取指阶段对应3条微指令

T_0 $PC \rightarrow MAR$ $1 \rightarrow R$

T_1 $M(MAR) \rightarrow MDR$ $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$ $OP(IR) \rightarrow$ 微地址形成部件

若一个 T 内安排一条微指令

所有控制信号来自控存中的微指令，如何读出这3条微指令？

则取指操作需3条微指令

$Ad(CMDR) \rightarrow CMAR$ 获取第2、3条取指微指令

$OP(IR) \rightarrow$ 微地址形成部件 $\rightarrow CMAR$ 第一条执行阶段的微指令的地址

此处与组合逻辑设计不同



(2) 取指阶段的微操作及节拍安排

若考虑需要 形成后续微指令的地址，取指阶段需要6条微指令

T_0 $PC \longrightarrow MAR$ $1 \longrightarrow R$

T_1 $Ad (CMDR) \longrightarrow CMAR$

T_2 $M (MAR) \longrightarrow MDR$ $(PC) + 1 \longrightarrow PC$

T_3 $Ad (CMDR) \longrightarrow CMAR$

T_4 $MDR \longrightarrow IR$ $OP (IR) \longrightarrow \text{微地址形成部件}$

T_5 $OP (IR) \longrightarrow \text{微地址形成部件} \longrightarrow CMAR$



(3) 执行阶段的微操作及节拍安排

此阶段也需要考虑形成后续微指令的地址

其中，取指微程序的入口地址 M，由执行阶段微程序的末尾微指令的下地址字段指出

- 非访存指令

- ① CLA 指令

$T_0 \quad 0 \longrightarrow AC$

$T_1 \quad Ad (CMDR) \longrightarrow CMAR$

- ② COM 指令

$T_0 \quad \overline{AC} \longrightarrow AC$

$T_1 \quad Ad (CMDR) \longrightarrow CMAR$



③ SHR 指令

$T_0 \quad L(AC) \longrightarrow R(AC) \quad AC_0 \longrightarrow AC_0$

$T_1 \quad \text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

④ CSL 指令

$T_0 \quad R(AC) \longrightarrow L(AC) \quad AC_0 \longrightarrow AC_n$

$T_1 \quad \text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

⑤ STP 指令

$T_0 \quad 0 \longrightarrow G$

$T_1 \quad \text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$



- 访存指令

- ⑥ ADD 指令

T_0 $Ad(IR) \longrightarrow MAR$ $1 \longrightarrow R$

T_1 $Ad(CMDR) \longrightarrow CMAR$

T_2 $M(MAR) \longrightarrow MDR$

T_3 $Ad(CMDR) \longrightarrow CMAR$

T_4 $(AC) + (MDR) \longrightarrow AC$

T_5 $Ad(CMDR) \longrightarrow CMAR$

- ⑦ STA 指令

T_0 $Ad(IR) \longrightarrow MAR$ $1 \longrightarrow W$

T_1 $Ad(CMDR) \longrightarrow CMAR$

T_2 $AC \longrightarrow MDR$

T_3 $Ad(CMDR) \longrightarrow CMAR$

T_4 $MDR \longrightarrow M(MAR)$

T_5 $Ad(CMDR) \longrightarrow CMAR$



⑧ LDA 指令

T_0 $\text{Ad (IR)} \longrightarrow \text{MAR}$ $1 \longrightarrow \text{R}$

T_1 $\text{Ad (CMDR)} \longrightarrow \text{CMAR}$

T_2 $\text{M (MAR)} \longrightarrow \text{MDR}$

T_3 $\text{Ad (CMDR)} \longrightarrow \text{CMAR}$

T_4 $\text{MDR} \longrightarrow \text{AC}$

T_5 $\text{Ad (CMDR)} \longrightarrow \text{CMAR}$



• 转移类指令

⑨ JMP 指令

$$T_0 \quad \text{Ad (IR)} \longrightarrow \text{PC}$$

$$T_1 \quad \text{Ad (CMDR)} \longrightarrow \text{CMAR}$$

⑩ BAN 指令

$$T_0 \quad A_0 \cdot \text{Ad (IR)} + \bar{A}_0 \cdot (\text{PC}) \longrightarrow \text{PC}$$

$$T_1 \quad \text{Ad (CMDR)} \longrightarrow \text{CMAR}$$

全部微操作 20个

微指令 38条



2. 确定微指令格式: 包括三个方面

(1) 微指令的编码方式

微操作个数少, 采用直接控制: 微指令操作控制字段某一位控制一个微操作

(2) 后续微指令的地址形成方式

由机器指令的操作码通过微地址形成部件形成

由微指令的下地址字段直接给出

(3) 微指令字长

由 20 个微操作

确定 操作控制字段 最少 20 位

由 38 条微指令

确定微指令的 下地址字段 为 6 位

微指令字长 可取 $20 + 6 = 26$ 位



(4) 微指令字长的确定

38 条微指令中有 19 条

是关于后续微指令地址 $\longrightarrow$ CMAR

其中 $\left\{ \begin{array}{ll} 1 \text{ 条} & \text{OP (IR)} \longrightarrow \text{微地址形成部件} \longrightarrow \text{CMAR} \\ 18 \text{ 条} & \text{Ad (CMDR)} \longrightarrow \text{CMAR} \end{array} \right.$

若把 Ad (CMDR) 直接送至控制存储器的地址线

则：省去了输至 CMAR 的时间，省去了 CMAR

同理： $\text{OP (IR)} \longrightarrow \text{微地址形成部件} \longrightarrow \text{控存地址线}$

两路地址可通过一个多路选择器切换使用

这样，可省去 19 条微指令，2 个微操作

$$38 - 19 = 19$$

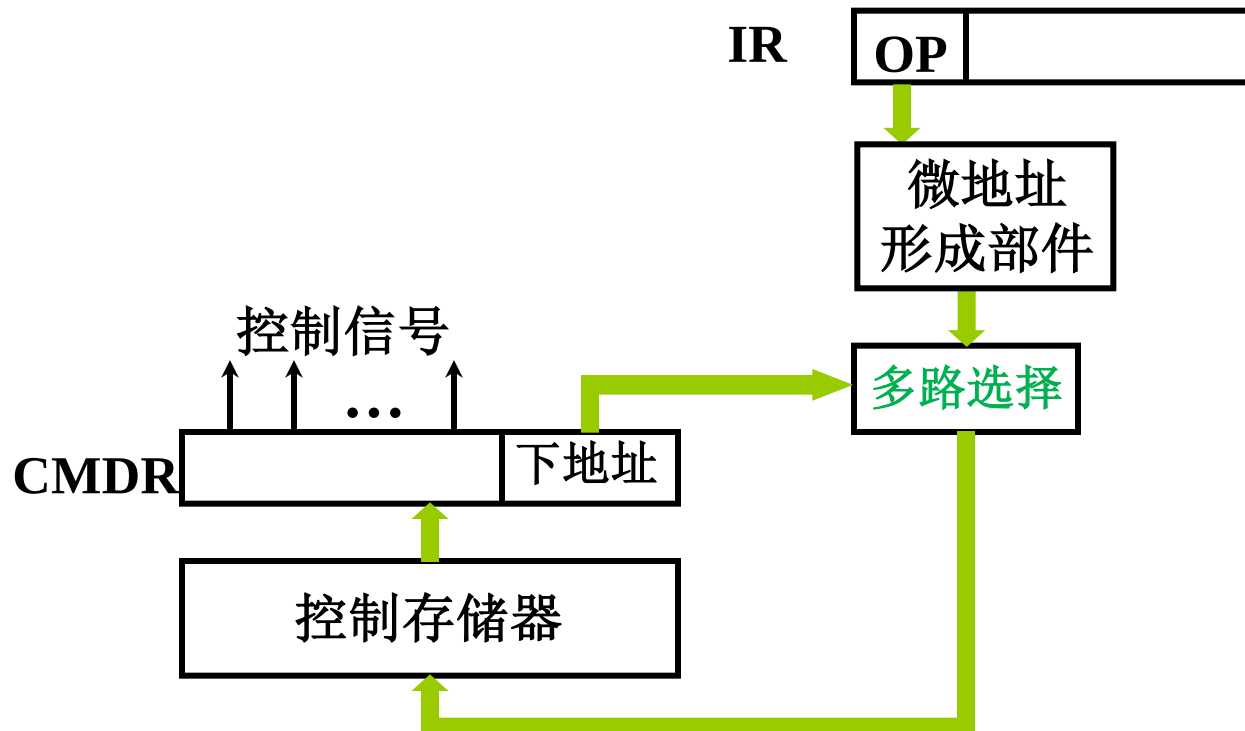
$$20 - 2 = 18$$

下地址字段最少取 5 位

操作控制字段最少取 18 位



(5) 省去了 CMAR 的控制存储器



考虑留有一定的余量 取操作控制字段 18 位 → 24 位
下地址字段 5 位 → 6 位 } 共 30 位

(6) 定义微指令操作控制字段每一位的微操作



第0位表示控制 $PC \rightarrow MAR$ 微操作

第1位表示控制 $1 \rightarrow R$ 微操作

第2位表示控制 $M(MAR) \rightarrow MDR$ 微操作

第3位表示控制 $(PC) + 1 \rightarrow PC$ 微操作

第4位表示控制 $MDR \rightarrow IR$ 微操作

.....

3. 编写微指令码点

微程序 名称	微指令 地址 (八进制)	微指令（二进制代码）														
		操作控制字段									下地址字段					
取指		0	1	2	3	4	...	10	...	23	24	25	26	27	28	29
	00	1	1								0	0	0	0	0	1
	01			1	1						0	0	0	0	1	0
	02					1					×	×	×	×	×	×
CLA	03										0	0	0	0	0	0
COM	04										0	0	0	0	0	0
ADD	10		1					1			0	0	1	0	0	1
	11			1							0	0	1	0	1	0
	12										0	0	0	0	0	0
LDA	16		1					1			0	0	1	1	1	1
	17			1							0	1	0	0	0	0
	20										0	0	0	0	0	0



作业

10.13,10.15

Thank you